

Introduction to Machine Learning

Unit 8 Problems: Support Vector Machines

Prof. Sundeep Rangan

1. Consider the data set for four points with features $\mathbf{x}_i = (x_{i1}, x_{i2})$ and binary class labels $y_i = \pm 1$.

x_{i1}	0	1	1	2
x_{i2}	0	0.3	0.7	1
y_i	-1	-1	1	1

- (a) Find a linear classifier that separates the two classes. Your classifier should be of the form

$$\hat{y} = \begin{cases} 1 & \text{if } b + w_1x_1 + w_2x_2 > 0 \\ -1 & \text{if } b + w_1x_1 + w_2x_2 < 0 \end{cases}$$

State the intercept b and weights w_1 and w_2 for your classifier. Note there is no unique answer as there are multiple linear classifiers that could separate the classes.

- (b) Find the maximum γ such that

$$y_i(b + w_1x_{i1} + w_2x_{i2}) \geq \gamma, \text{ for all } i,$$

for the classifier in part (a)?

- (c) Compute the margin of the classifier

$$m = \frac{\gamma}{\|\mathbf{w}\|}, \quad \|\mathbf{w}\| = \sqrt{w_1^2 + w_2^2}.$$

- (d) Which samples i are on the margin for your classifier?

Solution:

- (a) One possible linear classifier is $x_2 = 0.5$, which corresponds to: $b = -0.5, w_1 = 0, w_2 = 1$.
Verification: $(0, 0)$: $-0.5 < 0 \rightarrow y = -1$; $(1, 0.3)$: $-0.2 < 0 \rightarrow y = -1$; $(1, 0.7)$: $0.2 > 0 \rightarrow y = 1$; $(2, 1)$: $0.5 > 0 \rightarrow y = 1$.
- (b) For each point: $y_1(b + w_1x_{11} + w_2x_{12}) = -1(-0.5) = 0.5$; $y_2(b + w_1x_{21} + w_2x_{22}) = -1(-0.2) = 0.2$; $y_3(b + w_1x_{31} + w_2x_{32}) = 1(0.2) = 0.2$; $y_4(b + w_1x_{41} + w_2x_{42}) = 1(0.5) = 0.5$.
The maximum γ is $\gamma = 0.2$ (the minimum over all points).
- (c) $m = \frac{\gamma}{\|\mathbf{w}\|} = \frac{0.2}{\sqrt{0^2 + 1^2}} = 0.2$.
- (d) Samples $i = 2$ and $i = 3$ are on the margin (they achieve $y_i(b + w_1x_{i1} + w_2x_{i2}) = \gamma = 0.2$).

2. Consider the data set with scalar features x_i and binary class labels $y_i = \pm 1$.

x_i	0	1.3	2.1	2.8	4.2	5.7
y_i	-1	-1	-1	1	-1	1

Consider a linear classifier for this data of the form,

$$\hat{g} = \begin{cases} 1 & z > 0 \\ -1 & z < 0, \end{cases} \quad z = x - t,$$

where t is a threshold. For each threshold t , let $J(t)$ denote the sum hinge loss,

$$J(t) = \sum_i \epsilon_i, \quad \epsilon_i = \max(0, 1 - y_i z_i).$$

- Write a short python program to plot $J(t)$ vs. t for 100 values of t in the interval $t \in [0, 5]$.
- Based on the plot, what is one value of t that minimizes $J(t)$.
- For the value of t in part (b), find the corresponding slack variables ϵ_i .
- Which samples i violate the margin ($\epsilon_i > 0$) and which samples i are misclassified ($\epsilon_i > 1$).

Solution:

- The Python code to generate the plot is:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([0, 1.3, 2.1, 2.8, 4.2, 5.7])
y = np.array([-1, -1, -1, 1, -1, 1])
t_vals = np.linspace(0, 5, 100)
J_vals = []

for t in t_vals:
    z = x - t
    epsilon = np.maximum(0, 1 - y*z)
    J_vals.append(np.sum(epsilon))

plt.plot(t_vals, J_vals)
plt.xlabel('t')
plt.ylabel('J(t)')
plt.title('Hinge Loss vs Threshold')
plt.grid(True)
plt.show()
```

The resulting plot is shown in Figure 1.

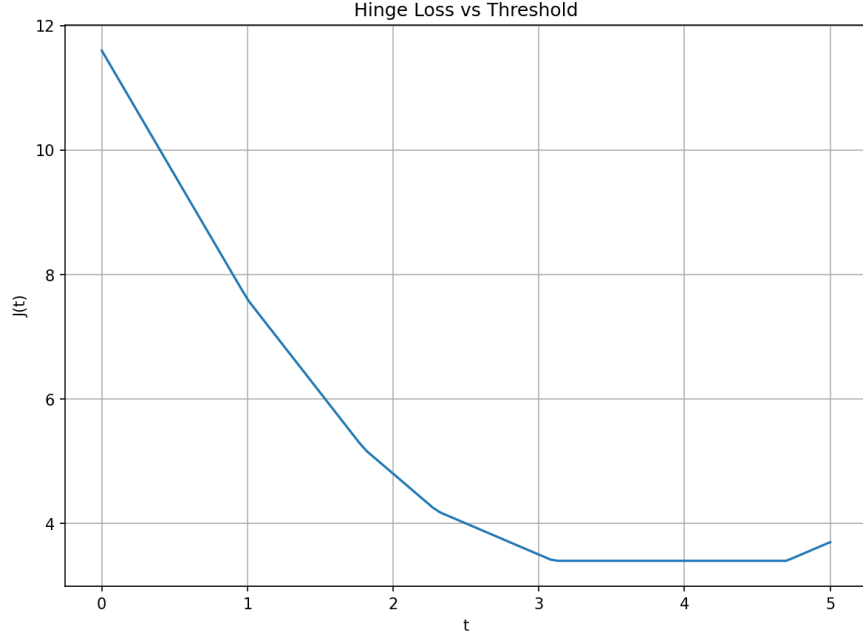


Figure 1: Plot of $J(t)$ vs. t for the hinge loss.

- (b) From the plot, one minimizing value is approximately $t = 2.45$.
- (c) For $t = 2.45$: $\epsilon_1 = \max(0, 1 - (-1)(0 - 2.45)) = \max(0, 1 - 2.45) = 0$; $\epsilon_2 = \max(0, 1 - (-1)(1.3 - 2.45)) = \max(0, 1 - 1.15) = 0$; $\epsilon_3 = \max(0, 1 - (-1)(2.1 - 2.45)) = \max(0, 1 - 0.35) = 0.65$; $\epsilon_4 = \max(0, 1 - (1)(2.8 - 2.45)) = \max(0, 1 - 0.35) = 0.65$; $\epsilon_5 = \max(0, 1 - (-1)(4.2 - 2.45)) = \max(0, 1 + 1.75) = 2.75$; $\epsilon_6 = \max(0, 1 - (1)(5.7 - 2.45)) = \max(0, 1 - 3.25) = 0$.
- (d) Samples $i = 3$, $i = 4$, and $i = 5$ violate the margin ($\epsilon_3 = 0.65 > 0$, $\epsilon_4 = 0.65 > 0$, $\epsilon_5 = 2.75 > 0$). Sample $i = 5$ is misclassified ($\epsilon_5 = 2.75 > 1$).
3. Consider an image recognition problem, where an image $\mathbf{X}$ and filter $\mathbf{W}$ are 4×4 matrices:

$$\mathbf{X} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad \mathbf{W} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

- (a) Recall that in linear classification, the 4×4 image matrices $\mathbf{X}$ and $\mathbf{W}$ can be represented as 16-dimensional vectors, $\mathbf{x} = \text{vec}(\mathbf{X})$ and $\mathbf{w} = \text{vec}(\mathbf{W})$ by stacking the columns of the matrices vertically. What are $\mathbf{x}$ and $\mathbf{w}$ for the matrices above.
- (b) What is the inner product $z = \mathbf{w}^\top \mathbf{x}$.
- (c) What is the inner product $z = \mathbf{w}^\top \mathbf{x}_{\text{right}}$ where $\mathbf{x}_{\text{right}}$ is the vector corresponding to the matrix $\mathbf{X}$ right shifted by one pixel with the left column filled with zeros.
- (d) What is the inner product $z = \mathbf{w}^\top \mathbf{x}_{\text{left}}$ where $\mathbf{x}_{\text{left}}$ is the vector corresponding to the matrix $\mathbf{X}$ left shifted by one pixel with the right column filled with zeros.

- (e) Write the python command that can convert a 4×4 image matrix, Xmat to the 16-dimensional vector, x. What is the python command to go from x to Xmat.

Solution:

- (a) Stacking columns vertically: $\mathbf{x} = [0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0]^T$;
 $\mathbf{w} = [0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0]^T$.
 (b) $z = \mathbf{w}^T \mathbf{x} = 0 \cdot 0 + 0 \cdot 0 + \dots + 1 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 + 1 \cdot 0 + \dots = 3$.
 (c) The right-shifted matrix is:

$$\mathbf{X}_{\text{right}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

so $\mathbf{x}_{\text{right}} = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0]^T$. $z = \mathbf{w}^T \mathbf{x}_{\text{right}} = 0$.

- (d) The left-shifted matrix is:

$$\mathbf{X}_{\text{left}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix},$$

so $\mathbf{x}_{\text{left}} = [0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]^T$. $z = \mathbf{w}^T \mathbf{x}_{\text{left}} = 0$.

- (e) $\mathbf{x} = \text{Xmat.flatten()}$ or $\mathbf{x} = \text{Xmat.ravel()}$ or $\mathbf{x} = \text{Xmat.reshape(16,)}$. To go back: $\text{Xmat} = \mathbf{x}.\text{reshape}(4, 4)$.

4. Consider the data set with scalar features x_i and binary class labels $y_i = \pm 1$.

x_i	0	1	2	3
y_i	1	-1	1	-1

A support vector classifier is of the form

$$\hat{y} = \begin{cases} 1 & z > 0 \\ -1 & z < 0, \end{cases} \quad z = \sum_i \alpha_i y_i K(x_i, x),$$

where $K(x, x')$ is the radial basis function, $K(x, x') = e^{-\gamma(x-x')^2}$, and $\gamma > 0$ and $\boldsymbol{\alpha} = [\alpha_1, \dots, \alpha_4]$ are parameters of the classifier.

- (a) Use python to plot z vs. x and $\hat{y}$ vs. x when $\gamma = 3$ and $\boldsymbol{\alpha} = [0, 0, 1, 1]$.
 (b) Repeat (a) with $\gamma = 0.3$ and $\boldsymbol{\alpha} = [1, 1, 1, 1]$.
 (c) Which classifier makes more errors on the training data.

Solution:

- (a) For $\gamma = 3$ and $\boldsymbol{\alpha} = [0, 0, 1, 1]$, the Python code is:

```

import numpy as np
import matplotlib.pyplot as plt

x_train = np.array([0, 1, 2, 3])
y_train = np.array([1, -1, 1, -1])
alpha = np.array([0, 0, 1, 1])
gamma = 3

x_plot = np.linspace(-1, 4, 500)
z = np.zeros_like(x_plot)

for i in range(len(x_train)):
    K = np.exp(-gamma * (x_train[i] - x_plot)**2)
    z += alpha[i] * y_train[i] * K

y_hat = np.sign(z)
y_hat[y_hat == 0] = 1

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(x_plot, z)
plt.axhline(y=0, color='k', linestyle='—', linewidth=0.5)
plt.xlabel('x')
plt.ylabel('z')
plt.title('z vs x (gamma=3, alpha=[0,0,1,1])')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(x_plot, y_hat)
plt.scatter(x_train, y_train, color='red', s=100, zorder=5)
plt.xlabel('x')
plt.ylabel('y_hat')
plt.title('y_hat vs x (gamma=3, alpha=[0,0,1,1])')
plt.grid(True)
plt.show()

```

The resulting plots are shown in Figure 2.

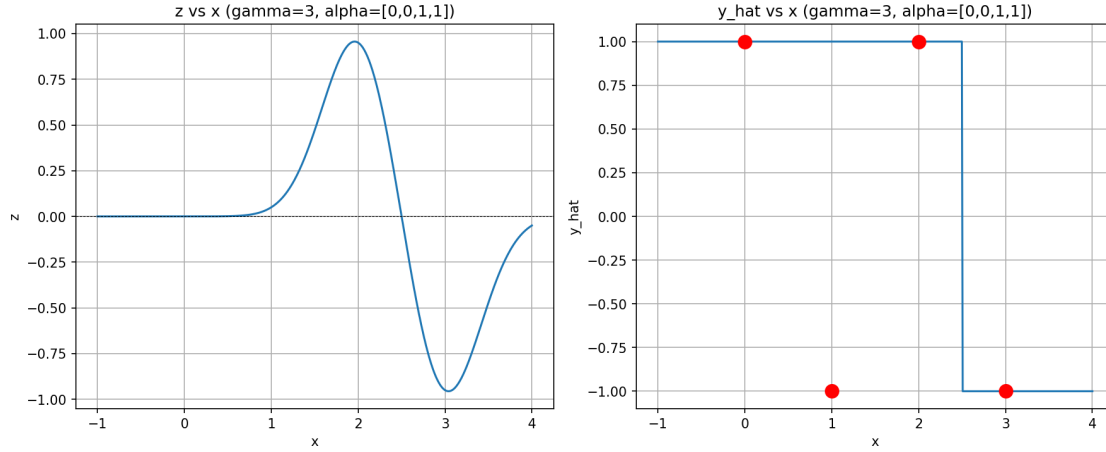


Figure 2: Plots of z vs. x and $\hat{y}$ vs. x for $\gamma = 3$ and $\alpha = [0, 0, 1, 1]$.

At the training points: $z(0) \approx 0.000006$, $z(1) \approx 0.0498$, $z(2) \approx 0.95$, $z(3) \approx -0.95$. So: $\hat{y}(0) = 1$ (correct), $\hat{y}(1) = 1$ (incorrect, should be -1), $\hat{y}(2) = 1$ (correct), $\hat{y}(3) = -1$ (correct). One error at $x = 1$.

(b) For $\gamma = 0.3$ and $\alpha = [1, 1, 1, 1]$, the Python code is:

```
alpha = np.array([1, 1, 1, 1])
gamma = 0.3

z = np.zeros_like(x_plot)
for i in range(len(x_train)):
    K = np.exp(-gamma * (x_train[i] - x_plot)**2)
    z += alpha[i] * y_train[i] * K

y_hat = np.sign(z)
y_hat[y_hat == 0] = 1

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(x_plot, z)
plt.axhline(y=0, color='k', linestyle='—', linewidth=0.5)
plt.xlabel('x')
plt.ylabel('z')
plt.title('z vs x (gamma=0.3, alpha=[1,1,1,1])')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(x_plot, y_hat)
plt.scatter(x_train, y_train, color='red', s=100, zorder=5)
plt.xlabel('x')
plt.ylabel('y_hat')
```

```
plt.title('y_hat vs x (gamma=0.3, alpha=[1,1,1,1])')
plt.grid(True)
plt.show()
```

The resulting plots are shown in Figure 3.

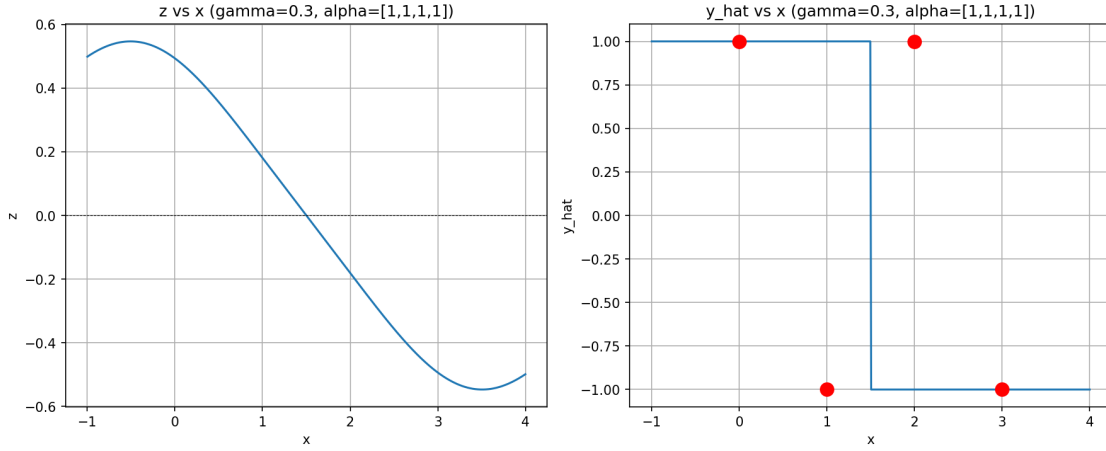


Figure 3: Plots of z vs. x and $\hat{y}$ vs. x for $\gamma = 0.3$ and $\alpha = [1, 1, 1, 1]$.

At the training points: $z(0) \approx 0.493$, $z(1) \approx 0.181$, $z(2) \approx 0.493$, $z(3) \approx -0.181$. So: $\hat{y}(0) = 1$ (correct), $\hat{y}(1) = 1$ (incorrect, should be -1), $\hat{y}(2) = 1$ (correct), $\hat{y}(3) = -1$ (correct). One error at $x = 1$.

(c) Both classifiers make 1 error on the training data at $x = 1$. They are equally accurate.